CSS234 WEB PROGRAMMING II

LECTURE 05 AUTHENTICATION & AUTHORIZATION

Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

Outline

- Basic Auth
- Cookies/Sessions
- Tokens (JWT)
- OAuth2/OIDC
- Passwordless

Authentication (AuthN) & Authorization (AuthZ)

Authentication

- Goal - Are you who you say you are?
- Factors
  - Something you know (Password)
  - Something you have (Phone)
  - Something you are (Biometrics)

Authorization

- Goal - What are you allowed to do?
- Techniques
  - RBAC (Role-Based Access Control)
  - ABAC (Attribute-Based)

HTTP is Stateless

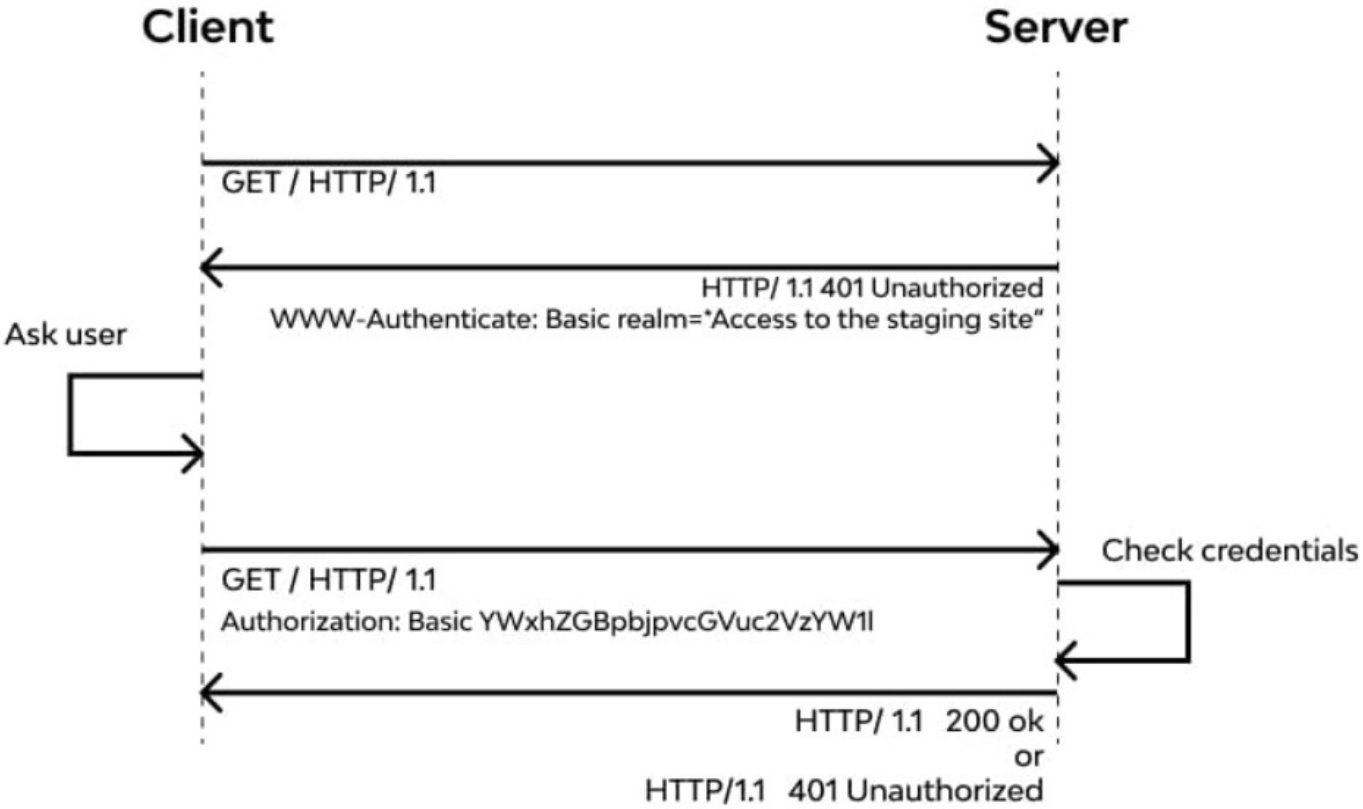
- The core problem - HTTP doesn't remember previous requests.
- Each HTTP request is independent; server can't tell if 2 requests came from the same browser or user.
- Web server applications maintain no information in memory from request to request (only information on disk survives from one request to another).
- Solution - Every request must carry some "Proof of Identity.”

Basic Authentication

- The `Authorization` Header.
- Basic `<Base64(username:password)>`

- **Vulnerabilities of Basic Auth**
  - No built-in logout.
  - Credential exposure if not using HTTPS.
  - Every request sends the password.

The Basic Auth Request-Response Life-Cycle



Basic Authentication

Get Headers - This request does NOT require authorization.

Get Headers - This request DOES require authorization.

HTTP Headers - struct	
Accept	text/html, */*; q=0.01
Accept-Charset	ISO-8859-1,utf-8;q=0.7,*;q=0.7
Accept-Encoding	gzip,deflate
Accept-Language	en-us,en;q=0.5
Connection	keep-alive
Content-Length	0
Cookie	
Host	www.bennadel.com
Keep-Alive	115
Referer	http://www.bennadel.com/temp/basic_authentication/test.cfm
User-Agent	
X-Original-URL	/temp/basic_authentication/authorize.cfm
X-Rewrite-URL	/temp/basic_authentication/authorize.cfm

No Authorization Header has been passed.

Get Headers - This request does NOT require authorization.

Get Headers - This request DOES require authorization.

HTTP Headers - struct	
Accept	text/html, */*; q=0.01
Accept-Charset	ISO-8859-1,utf-8;q=0.7,*;q=0.7
Accept-Encoding	gzip,deflate
Accept-Language	en-us,en;q=0.5
Authorization	Basic dXNlcmShbWU6cGFzc3dvcmQ=
Connection	keep-alive
Content-Length	0
Cookie	
Host	www.bennadel.com
Keep-Alive	115
Referer	http://www.bennadel.com/temp/basic_authentication/test.cfm
User-Agent	
X-Original-URL	/temp/basic_authentication/authorize.cfm?use401Response=1
X-Rewrite-URL	/temp/basic_authentication/authorize.cfm?use401Response=1

The Authorization header was passed AFTER a 401 status code was returned.

Console HTML CSS Script DOM Net

Clear Persist Profile All Errors Warnings Info Debug Info

GET http://username:password@www.b se=1 401 Unauthorized 100ms

GET http://username:password@www.b se=1 401 Unauthorized 146ms

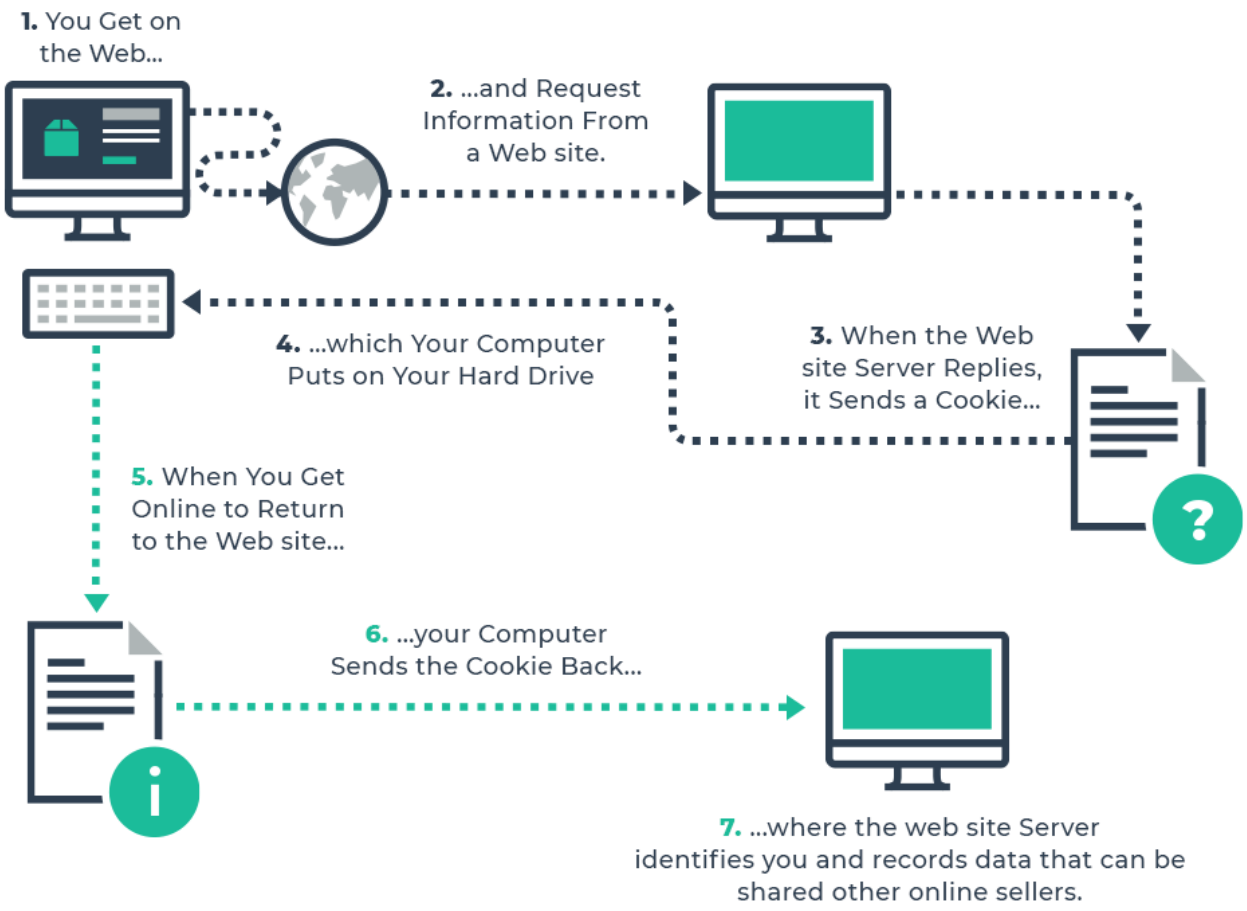
Cookies: state in a stateless world

- A small piece of data that a server sends to the user's web browser.
- The browser may store it and send it back with later requests to the same server.
- Typically, it's used to tell if two requests came from the same browser — keeping a user logged-in.
  - It remembers stateful information for the stateless HTTP protocol.

Cookies: state in a stateless world

Cookie basics:

- The first time a browser connects with a particular server, there are no cookies.
- When the server responds it includes a Set-Cookie: header that defines a cookie.
- Each cookie is just a name-value pair.
- In the future whenever the browser connects with the same server, it includes a Cookie: header containing the name and value, which the server can use to connect related requests.



devopedia.org

Cookies: state in a stateless world

Cookies are mainly used for three purposes:

- **Session management**
  - Logins, shopping carts, game scores, or anything else the server should remember
- **Personalization**
  - User preferences, themes, and other settings
- **Tracking**
  - Recording and analyzing user behavior

What is a Session

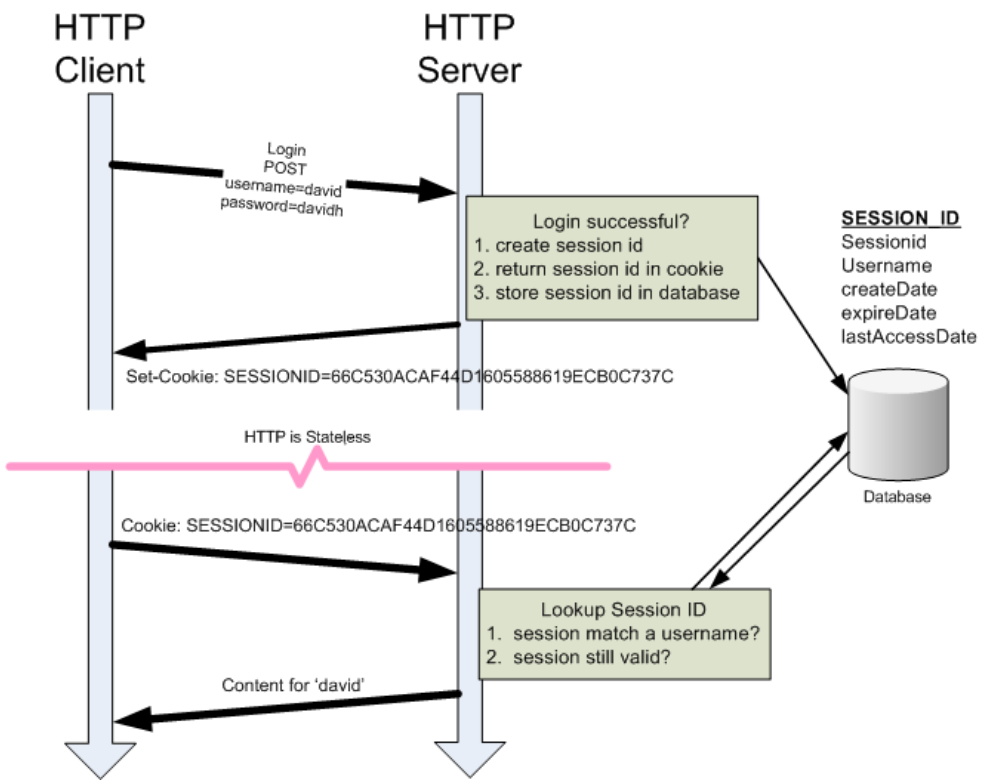
- A mechanism for maintaining state
  - For the particular user and the particular web app
  - Within the server, independent of the browser
- This notion of state is called a session
  - Information for a particular user
  - Information for a particular application
  - Information for a particular use of the application

Session Properties

- What information needs to be kept with the session

- Sample information
  - User id if one exists
  - Host, last-used time
  - Shopping cart
    - Associated with user id?
    - How to handle log in afterwards
  - Input values for forms (to be (re)filled automatically)
  - Previous searches or history
  - Site customization values

Tracking Sessions



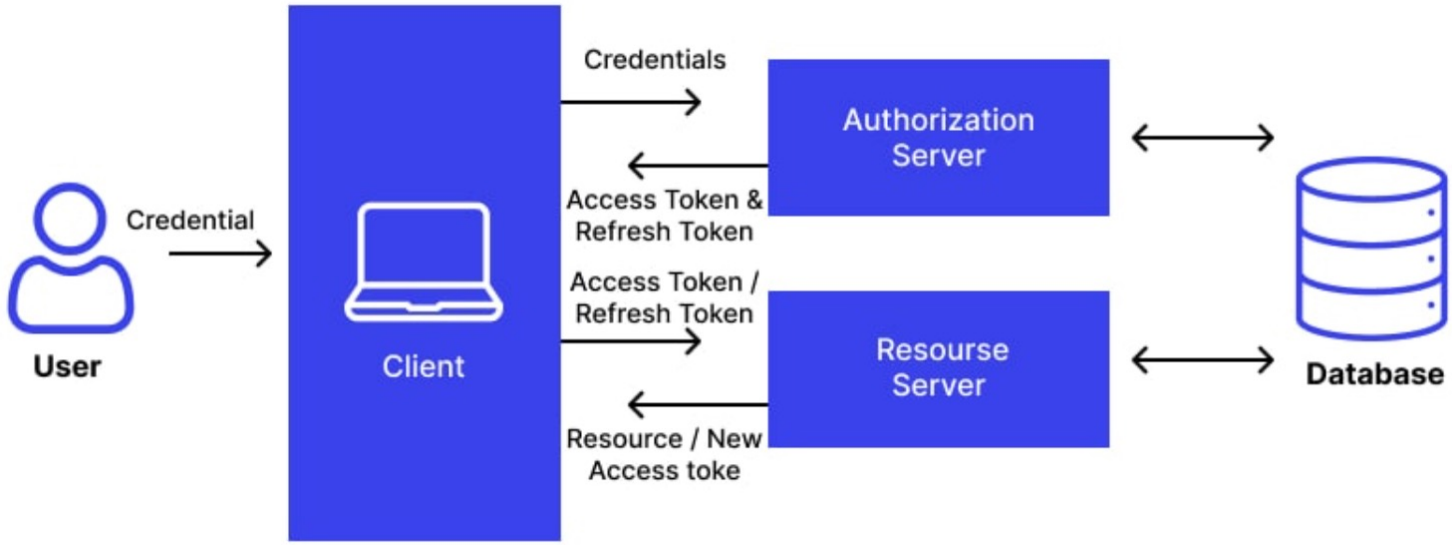
Session Identifiers

- How much information needs to be conveyed
  - Really only need one piece of data
    - Use this as an index to a table (or database) on the server
    - Table holds all the information related to the session
  - This is the **session ID**
- Tracking Session Ids
  - Ensure validity
  - Ensure it is coming from same machine
  - Time out if not used for certain amount of time

Token-Based Auth

- A security technique where users verify their identity and receive a unique access token.
- Token - A string representing user identity, usually signed cryptographically.
- The token acts as a "temporary digital key."
- Decouples the authentication process from the server's session state.

Token-based Authentication in action



Request - User provides credentials.

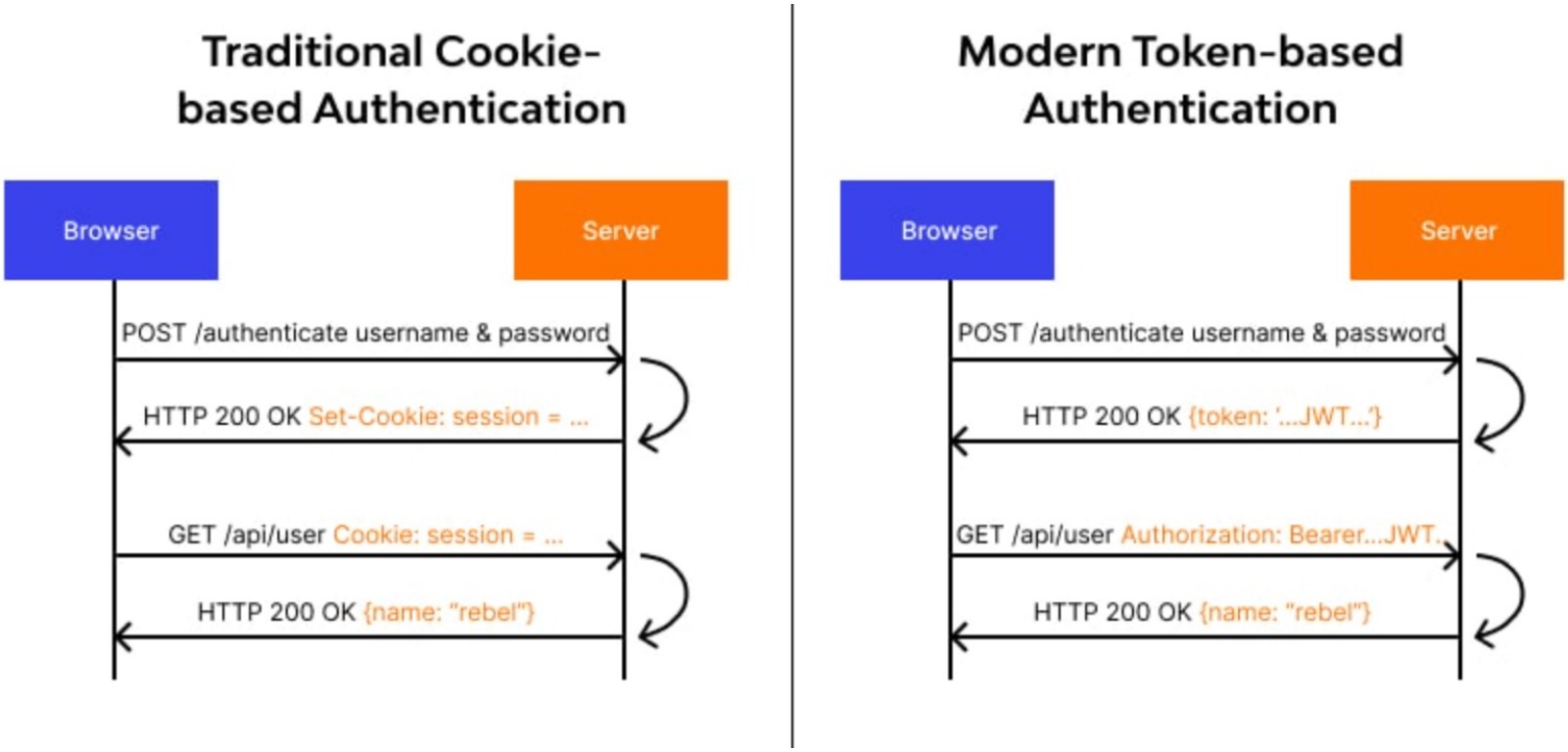
Verification - Server checks the database.

Token Issue - Server generates a signed token.

Storage - Client stores the token (e.g., `LocalStorage, SessionStorage`).

Access - Client sends the token in the Authorization header for every request.

Traditional vs Modern Auth



Types of Tokens

Hardware Tokens (Physical Security)

- Physical devices (USB, Key fobs) used for MFA/2FA.
- The user physically connects the device to the system to prove possession.
- Categories - USB tokens, Remote tokens, and Key fobs.
- Adds a physical layer of security that software cannot replicate.

JWT (JSON Web Tokens) - The Open Standard

- Standard - RFC 7519.
- Self-contained; no need to query a database to validate the user.
- Can be sent via URL, POST parameters, or HTTP Headers.

Types of Tokens

One-Time Password (OTP) Tokens

- Generates single-use numeric codes (4-12 digits).
- Authenticator Apps or SMS.
- Generation Methods -
  - Synchronous - Based on time and a private key (TOTP).
  - Asynchronous - Challenge-Response Authentication Mechanism (CRAM).

API Tokens (Application Identification)

- Unique identifiers for applications requesting access to a service.
- Safer than sending username/password combinations over HTTP.
- Frequently used in **OAuth2** as "Access Tokens."

Why Modern Apps Prefer Tokens?

- **Statelessness -** Server doesn't need to keep a record of which users are logged in.
- **Scalability -** Perfect for Microservices where multiple servers need to verify the user independently.
- **Mobile-Ready -** Cookies are often difficult to manage in native mobile apps; tokens are much simpler.
- **Cross-Domain/CORS -** Easier to use across different domains than traditional cookies.

Security Perspective

- Most common type - Bearer Token.
  - "The bearer of this token is granted access."
  - Warning: If an attacker gets the token, they ARE the user.

- Code Example (HTTP Header)

```
GET /api/v1/profile HTTP/1.1
Host: api.example.com
Authorization: Bearer <your_access_token_here>
```

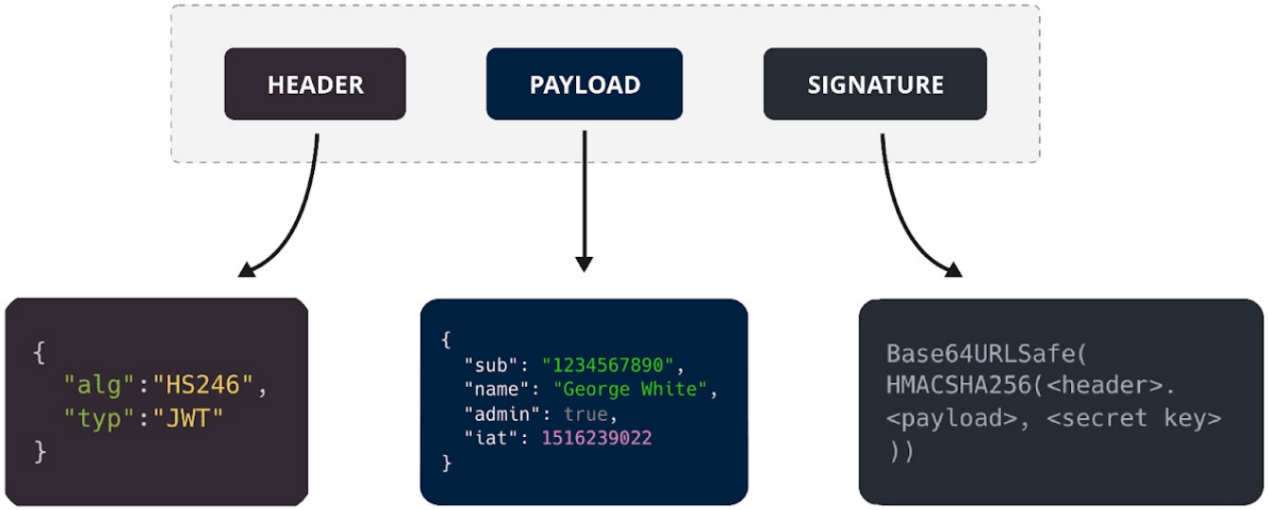
- Token Revocation Challenge
  - Since the server is stateless, it's hard to "kill" a token before it expires.
  - Solution: Blacklisting (storing revoked tokens in Redis) or Short expiration times with Refresh Tokens.

JWT: JSON Web Token

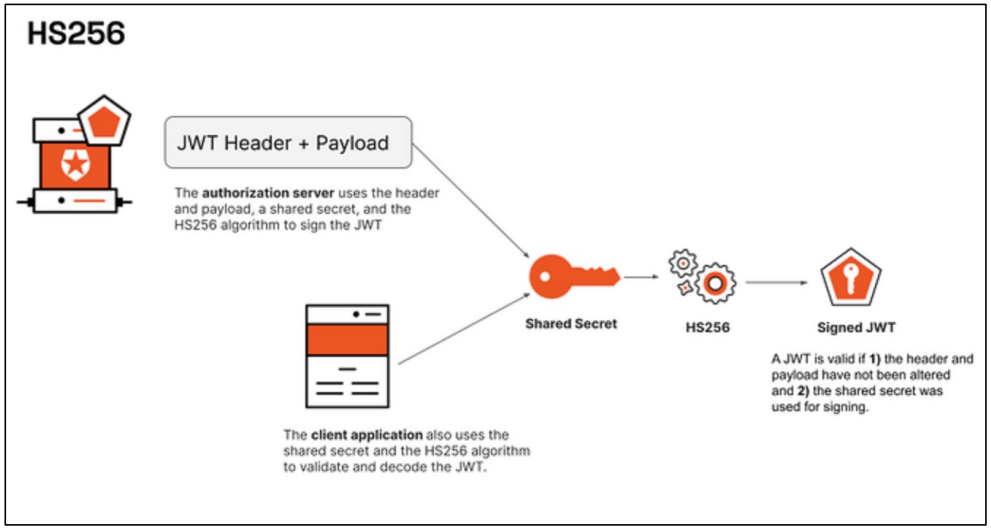
- An open specification (RFC 7519) that describes a compact and self-contained JSON object for the secure exchange of information between parties.
- JWTs may be signed with a secret (using the HMAC algorithm) or with a public/private key pair using RSA or ECDSA.

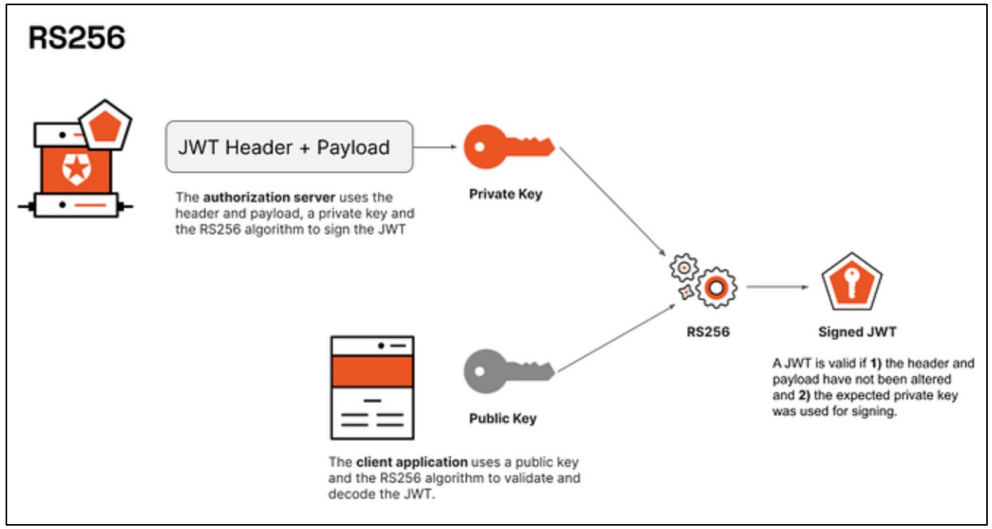
Anatomy

- Algorithm and type
- Data (Claims)
- Verification part

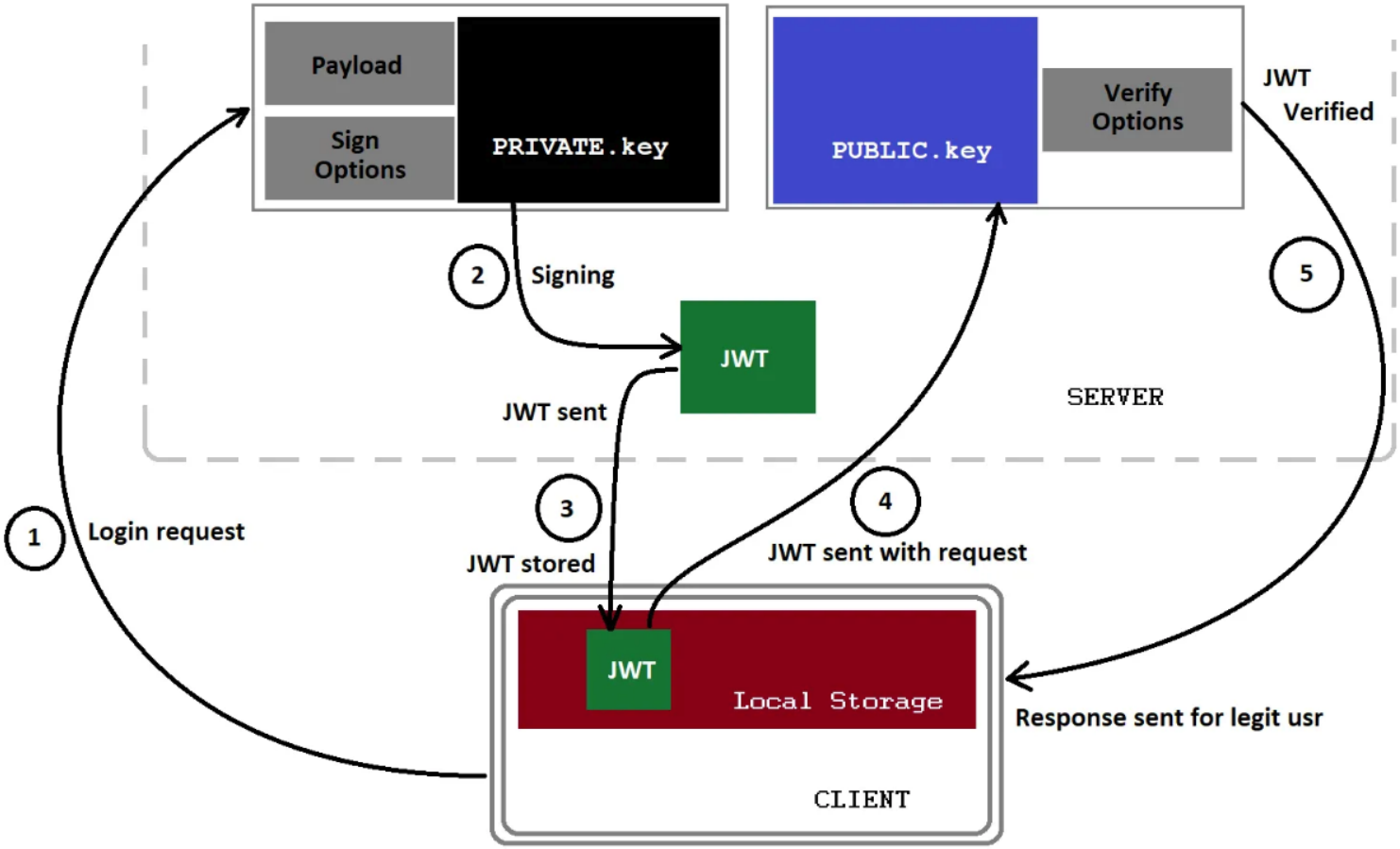


Symmetric vs Asymmetric Signing





The Flow of Token-Based Auth



JWT Security

Refresh Tokens

- Access Token: Short-lived (15 min).
- Refresh Token: Long-lived (7 days), used to get new Access Tokens.

Common Mistakes with JWT

- Storing sensitive data in payload, using weak secrets, not checking the alg header properly.

OAuth 2.0

- The industry-standard framework for Authorization.
- It allows third-party applications to gain limited access to a user's resources (like photos or contact lists) without requiring the user's password.

Key Actors in OAuth

- Resource Owner - The User.
- Client - The Application.
- Authorization Server - Google/Facebook.
- Resource Server - The API holding the data.

OAuth 2.0 Core roles

Role	Description
Resource Owner	The user who owns the data and grants access to it.
Client	The application requesting access to the user's resources.
Authorization Server	The server that authenticates the user and issues access tokens.
Resource Server	The server hosting the protected data (e.g., Google Drive API).

Key Grant Types (Flows)

The "flow" is the set of steps an application takes to get an access token.

Authorization Code Flow

- The most secure flow for web apps
- It uses a temporary "code" exchanged for a token on the server side.

Client Credentials Flow

- Used for machine-to-machine (M2M) communication where no user is present.

Authorization Code with PKCE

- The current security standard for mobile and Single Page Apps (SPA) to prevent code interception.

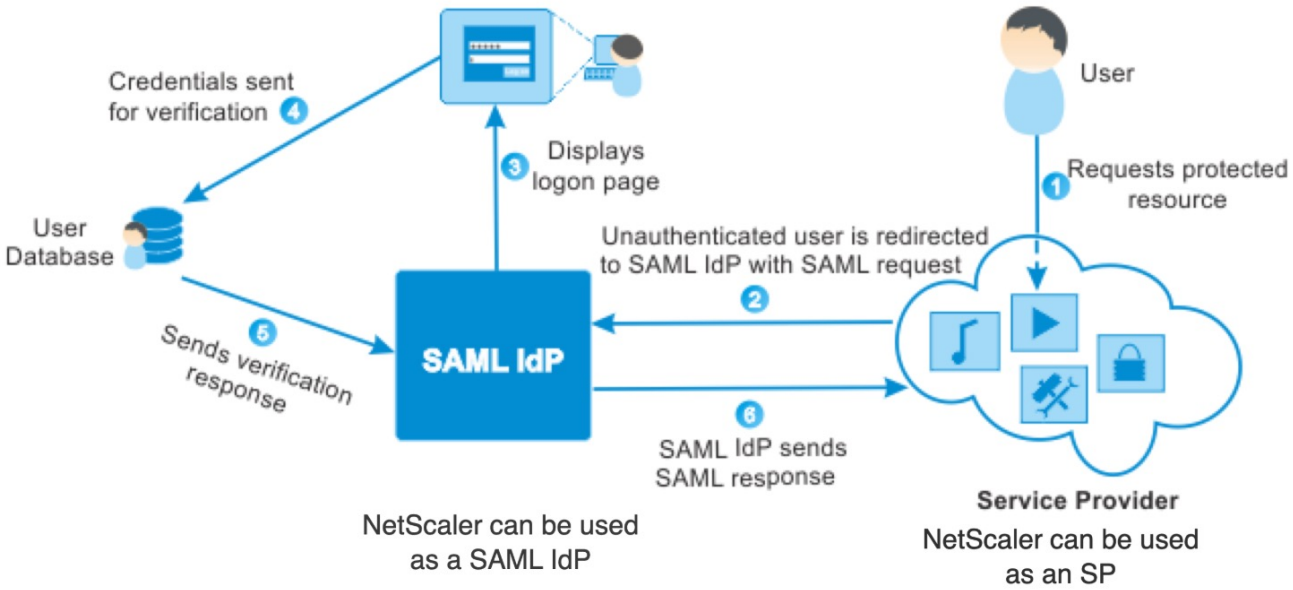
Device Flow

- Designed for devices with limited input, like Smart TVs or game consoles.

SAML 2.0

- SAML 2.0 (Security Assertion Markup Language) is an XML-based open standard used for exchanging authentication and authorization data between an Identity Provider (IdP) and a Service Provider (SP).
- It is the "gold standard" for enterprise Single Sign-On (SSO).
- SAML 2.0 allows users to access multiple web applications using just one set of credentials.
- Instead of the application (SP) handling your password, it trusts a central authority (IdP) to verify who you are.
- This verification is passed as a digitally signed XML document called a SAML Assertion.

How SAML 2.0 works

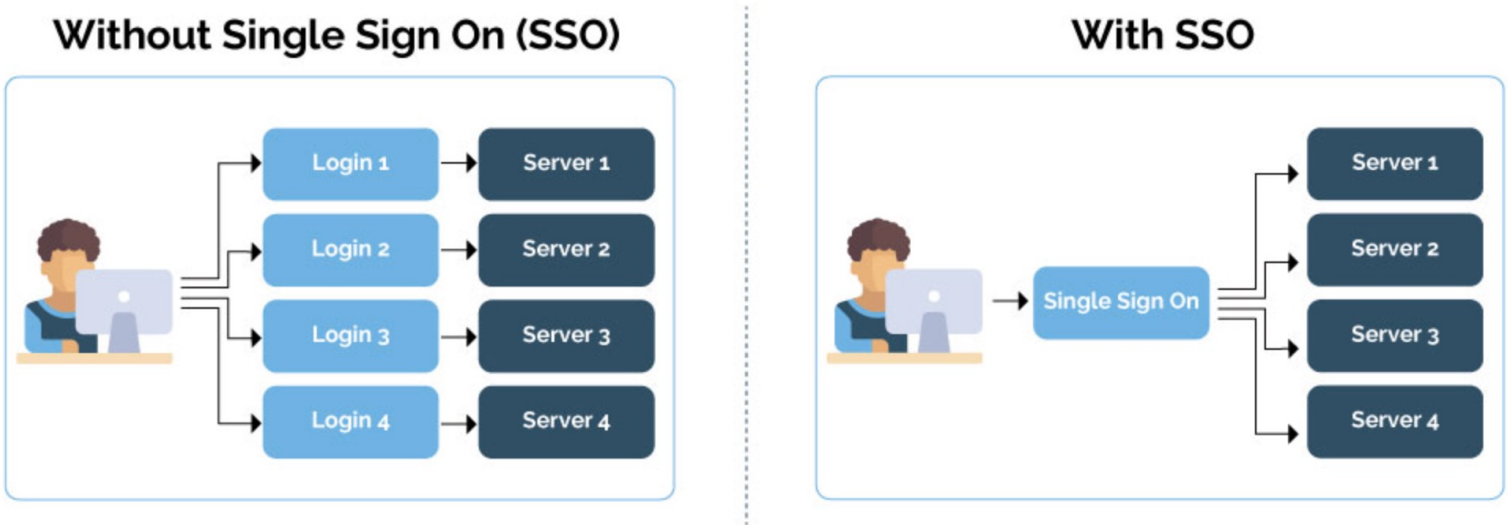


Key Features of SAML 2.0

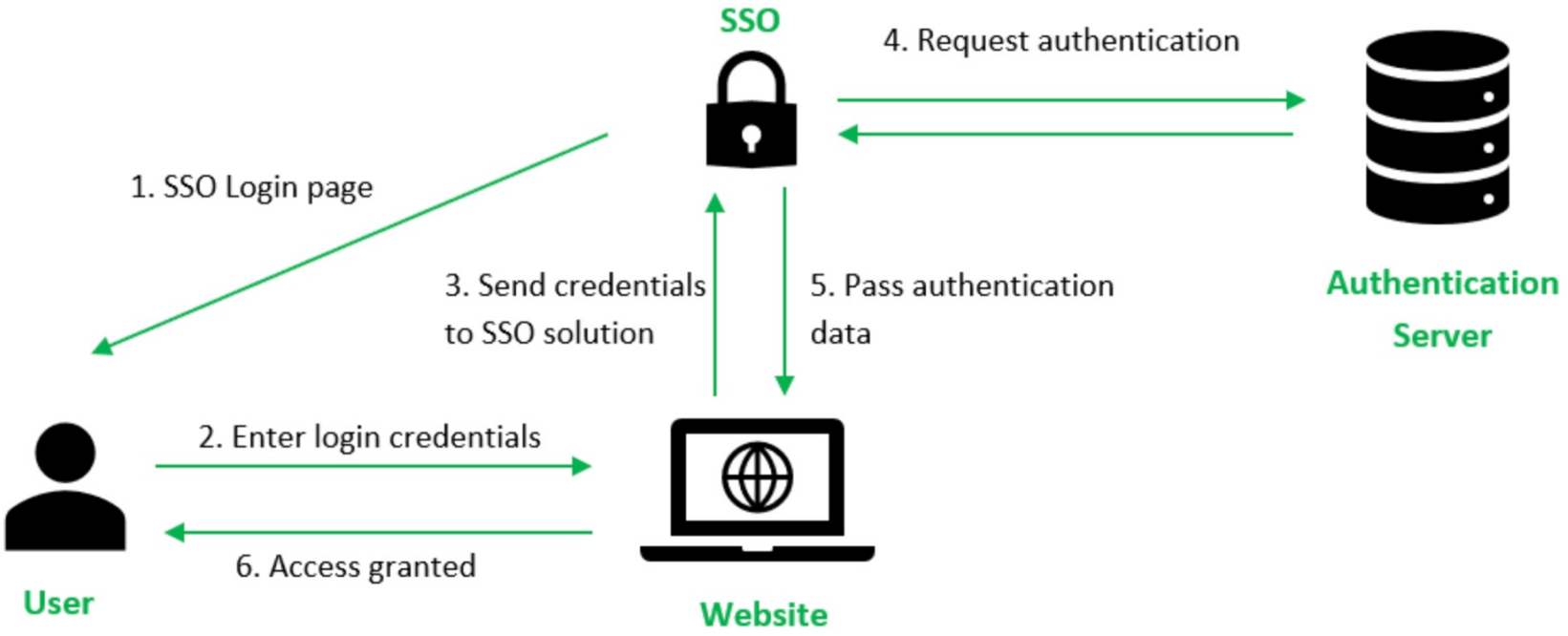
- **Single Sign-On (SSO) -** Users log in once and gain access to all connected systems without re-entering passwords.
- **Single Logout (SLO) -** Logging out of one application can automatically terminate sessions across all other applications in the federation.
- **Strong Security (XML Signatures/Encryption) -** Assertions are digitally signed and can be encrypted to ensure data integrity and confidentiality.
- **Interoperability -** It works across different platforms and vendors, making it ideal for business-to-business (B2B) integrations.
- **Just-in-Time (JIT) Provisioning -** Automatically creates a user account in the application the first time they log in via SAML.

Single Sign-On (SSO)

- A centralized authentication framework that allows a user to verify their identity once and gain access to multiple independent software systems without re-entering credentials.



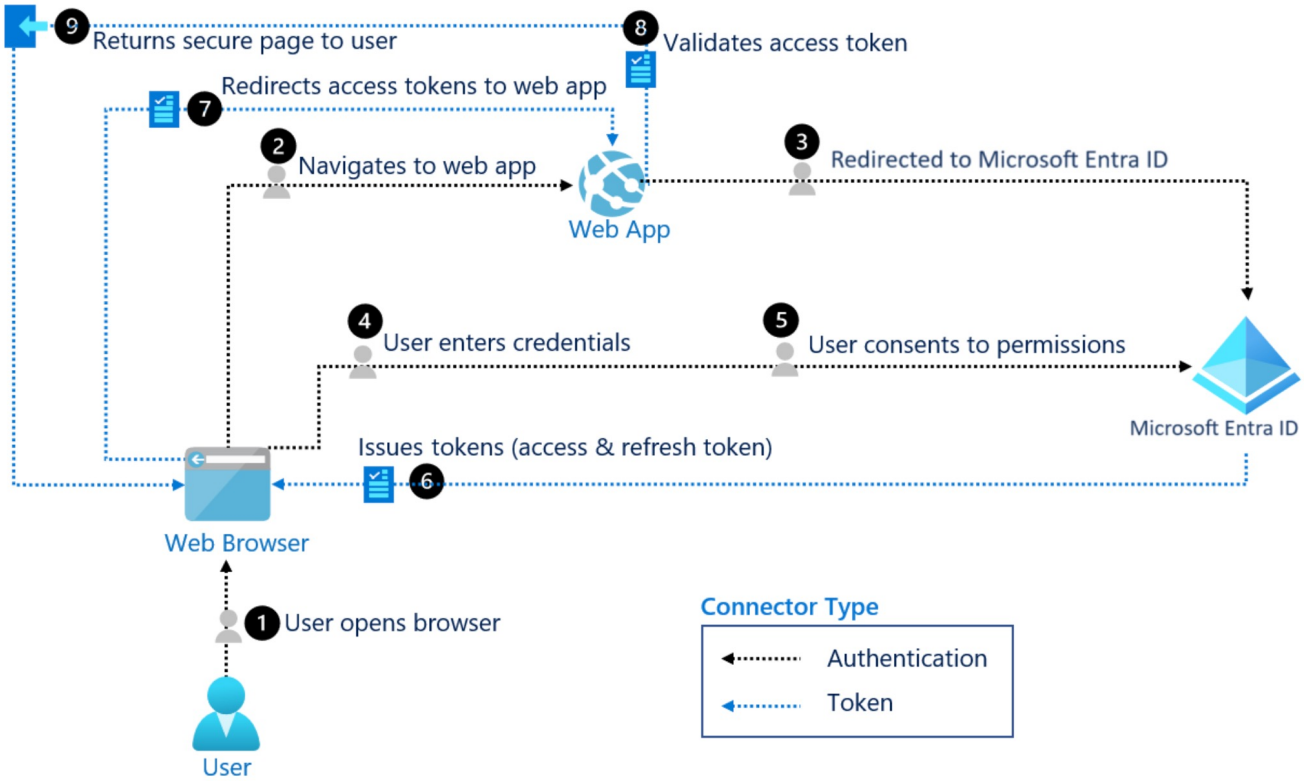
Single Sign-On (SSO)



OpenID Connect (OIDC)

- A simple authentication protocol built on top of OAuth 2.0.
- It lets apps verify who you are without needing your password, using a trusted identity provider like Google or Microsoft.
- Instead of separate logins for every site, OIDC enables single sign-on (SSO), where one login grants access to many services securely via ID tokens.

OIDC


9 Returns secure page to user
8 Validates access token
Redirects access tokens to web app
7
2 Navigates to web app
3 Redirected to Microsoft Entra ID
Web App
4 User enters credentials
5 User consents to permissions
Microsoft Entra ID
Issues tokens (access & refresh token)
6
Web Browser
1 User opens browser
User
Connector Type
Authentication
Token

OIDC vs. SAML Comparison

Key Differences

	OIDC	SAML
Technology	JSON Web Tokens (JWT) and REST APIs (lightweight)	XML assertions (heavy)
Best Use Case	Mobile apps, Single-Page Applications (SPAs), and APIs	Enterprise-grade SaaS and legacy applications
Complexity	Developer-friendly with easier integration	More complex to implement
Security	Secure but relies on modern token-based security	Highly secure, often preferred by large organizations
Performance	Better performance for modern, distributed systems	Good performance

When to Choose Which

Choose OIDC for

- New applications, mobile apps, modern web frameworks
- API-heavy environments

Choose SAML for

- Enterprise B2B SaaS
- Legacy systems that do not support modern protocols, and strict compliance environments that already use SAML.

- While SAML is more mature, OIDC is increasingly becoming the default standard for modern development.

Passwordless authentication

- It lets users log in without entering passwords, using alternatives ways instead.
- Security boosts by cutting phishing and credential-stuffing risks while speeding up access.

How It Works

- Users start login by entering an email, username, or scanning a QR code.
- The system sends a one-time link, push notification, or prompts biometrics.
- A private key on the device signs a challenge from the server, which verifies it against the public key.
- Standards like FIDO2/WebAuthn enable this cryptographic proof across browsers and apps.

Common Methods

- Biometrics - Fingerprint, face, or voice ID unlocks a device-bound key.
- Magic links/OTPs - Email or SMS with a clickable link or code.
- Security keys - Hardware like YubiKey or phone passkeys (FIDO2).
- Push/QR - Approve via app notification or scan.

Security & Best Practices

Storing Passwords Securely

- NEVER store plain text
- Use Argon2 or BCrypt with high-cost factors
- Salt + Hash

Multi-Factor Authentication (MFA)

- OTP
- Authenticator Apps (TOTP)
- FIDO2/WebAuthn (Biometrics)

Homework

Scenario: You are developing a REST API for a mobile application. The server should not store session state. Instead, it should issue a Token that the client must send with every request. Additionally, you need to handle User Roles (Admin vs. User).

Requirements:

- Use jsonwebtoken package.
- Create a **POST /login** route:
  - Accepts username.
  - If username is "admin", sign a token with payload { role: 'admin' }.
  - Otherwise, sign a token with payload { role: 'user' }.
  - Return the token to the client.
- Create an **Authentication Middleware**:
  - Extracts the token from the Authorization: Bearer <token> header.
  - Verifies the token. If valid, attach the payload to req.user.
- Create an **Authorization Middleware (checkAdmin)**:
  - Checks if req.user.role is exactly 'admin'.
  - If not, return status 403 Forbidden.
- Create a protected route **/admin-only** that uses both middlewares.